

Scheduling: Multi-level Feedback Queue

CS232 Operating Systems

Fall 2019

Taj Muhammad Khan

Revision

- Which of them:
 - Is fair
 - Favours small, I/O oriented jobs jobs
 - Favours big, CPU-intensive jobs
 - Have good response times
- FIFO
- SJF
- STCF
- RR

No prior knowledge

- Relax constraint no. 5 from previous chapter
- Ideally, we would like to:
 - Optimize turnaround time
 - Minimize response time
- If the scheduler has no prior knowledge, then it must learn.
- How?
- From history!

Multi-level Feedback Queue

- Multiple queues
- Each with a different priority level
- A job can be only on one queue at any given moment
- Jobs with higher priority run first
If $P(A) > P(B)$, A runs, B doesn't
- Jobs with same priority run in RR
If $P(A) == P(B)$, both A & B run in RR

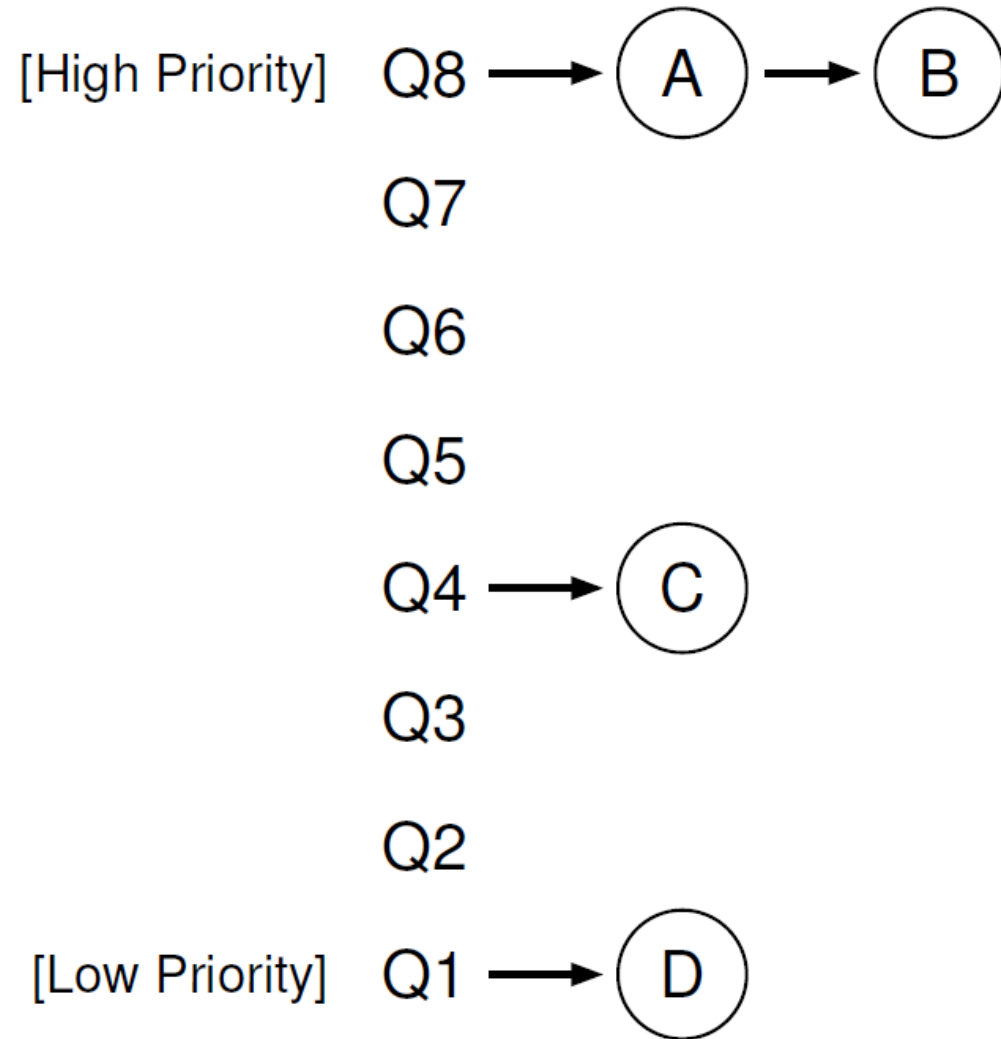


Figure 8.1: **MLFQ** Example

MLFQ contd.

- MLFQ varies the priority of a job based on observed behavior
- If a job repeatedly relinquishes CPU, its priority is kept high
- If a job is CPU intensive and hogs CPU, its priority is lowered
- When a job enters the system, its priority is highest
- If a job uses up its entire time slice, its priority is reduced
- If a job gives up CPU before its time slice is up, it stays at same priority

Examples: Single long running job

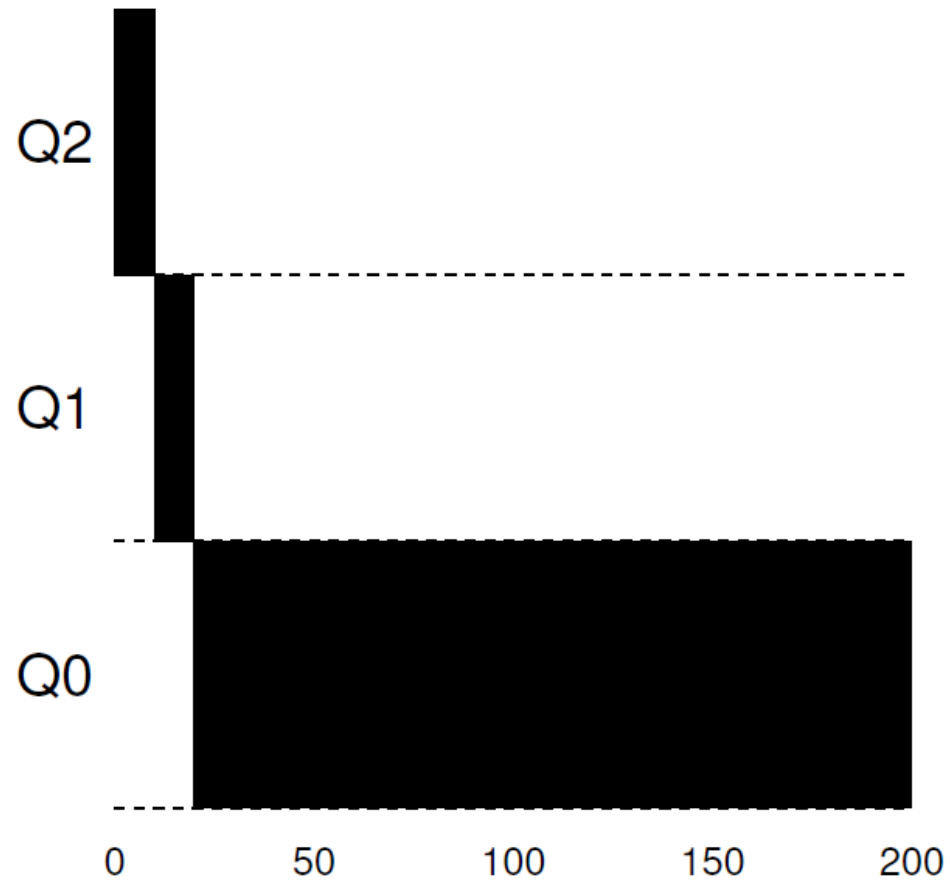


Figure 8.2: Long-running Job Over Time

Example: A short job joins later

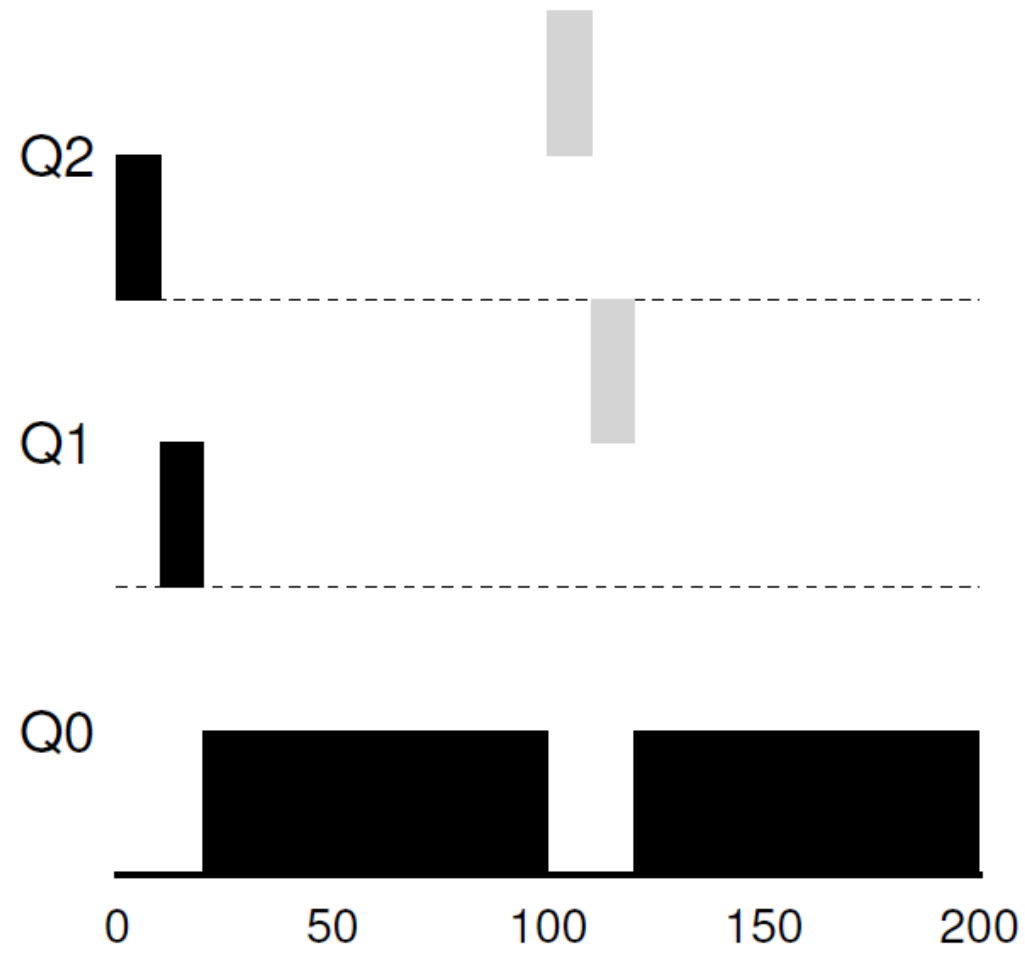


Figure 8.3: *Along Came An Interactive Job*

Examples: CPU-intensive + I/O intensive jobs

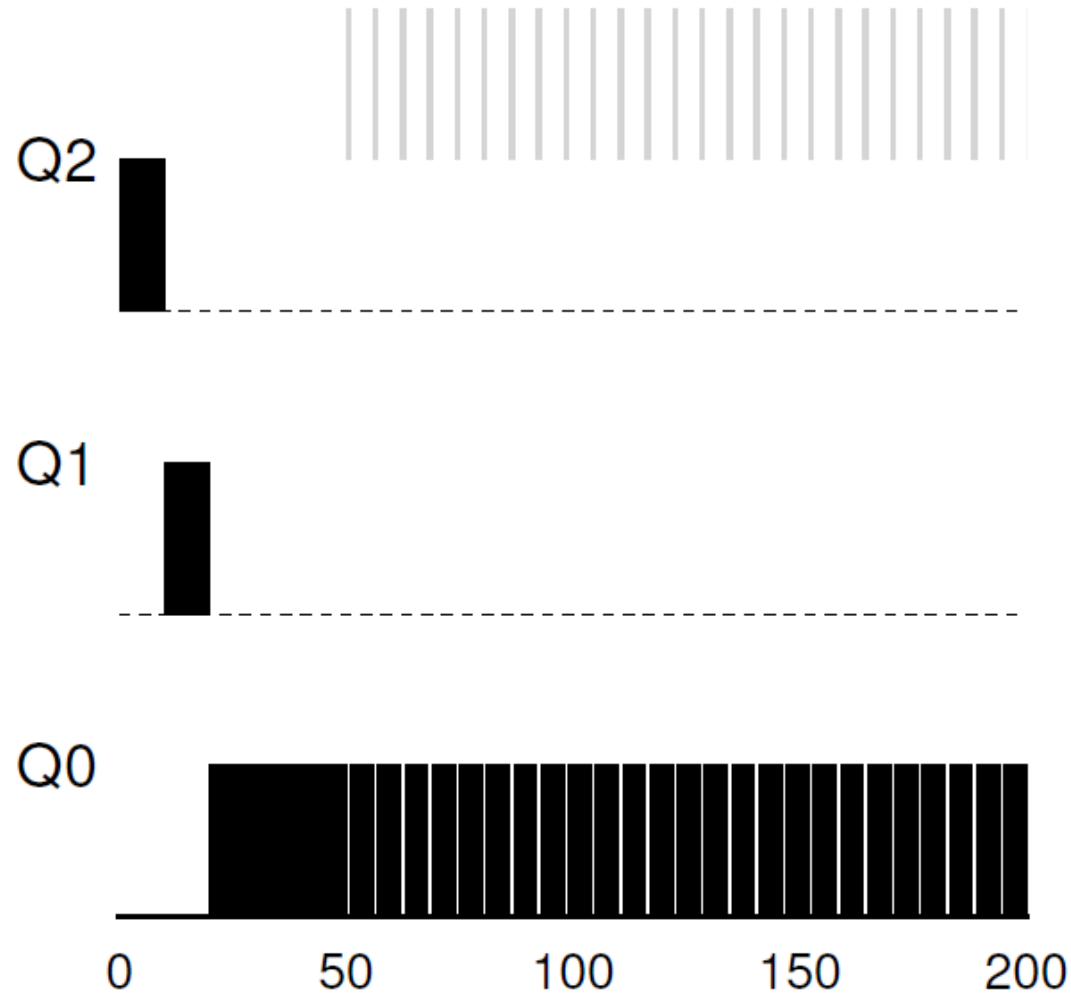


Figure 8.4: A Mixed I/O-intensive and CPU-intensive Workload

How about the metrics?

- Turn around time
- Response time
- Are we fair?

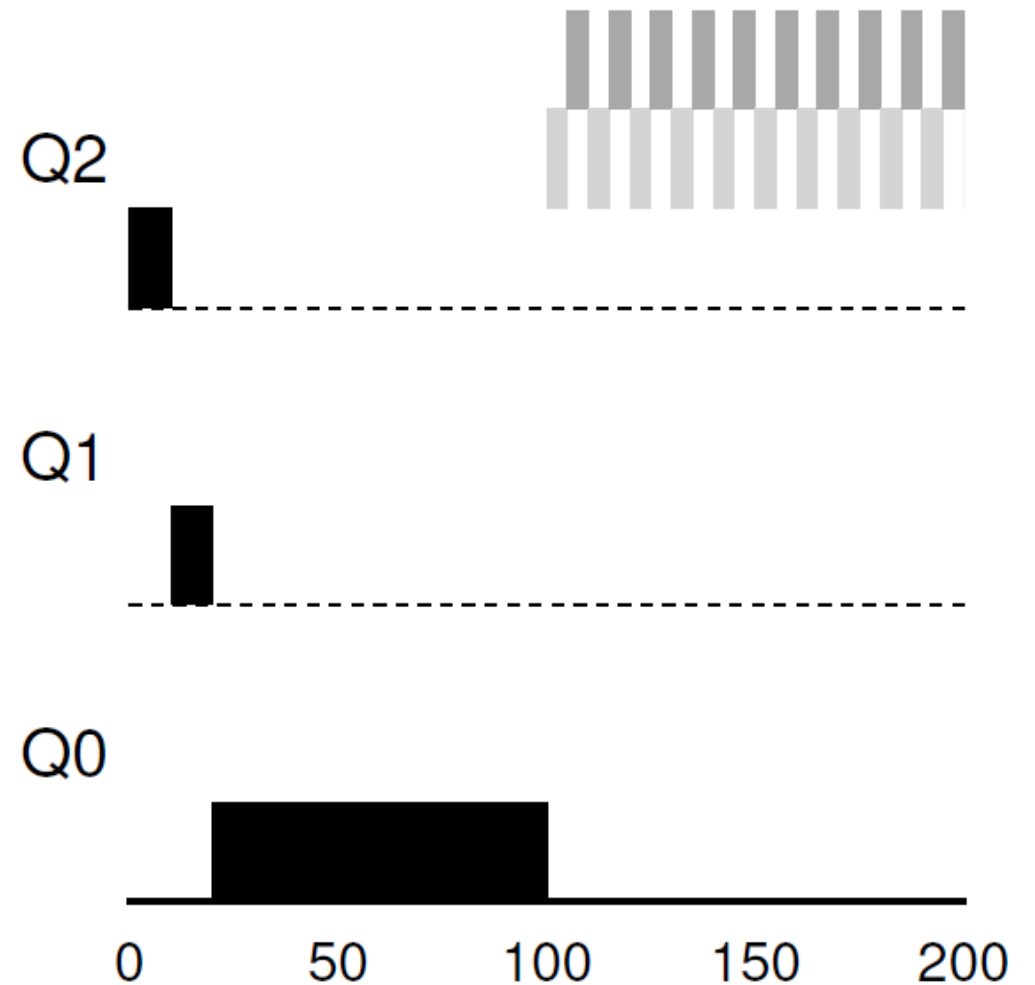
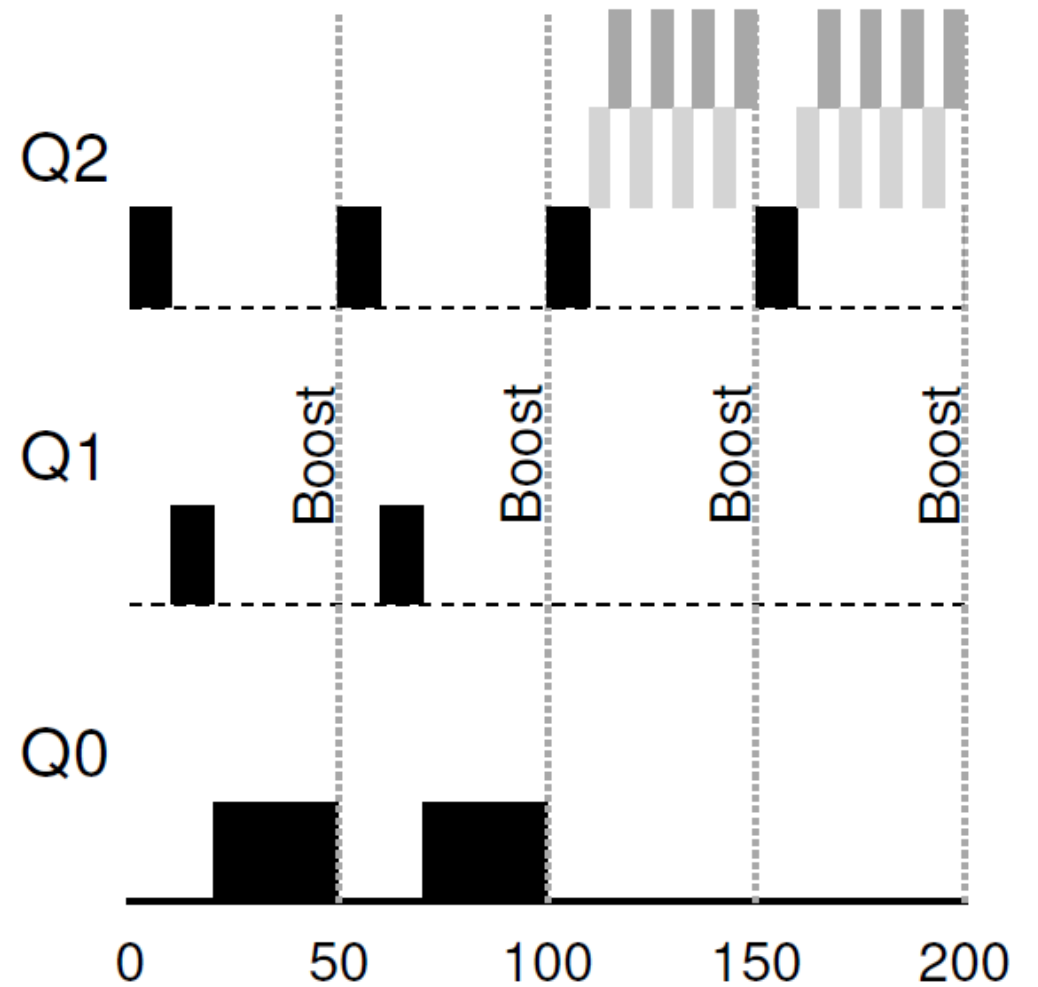


Figure 8.5: Without (Left) and

Solution: Priority boost

After a certain period S ,
bump all jobs to top
queue



d With (Right) Priority Boost

What about smart kids?

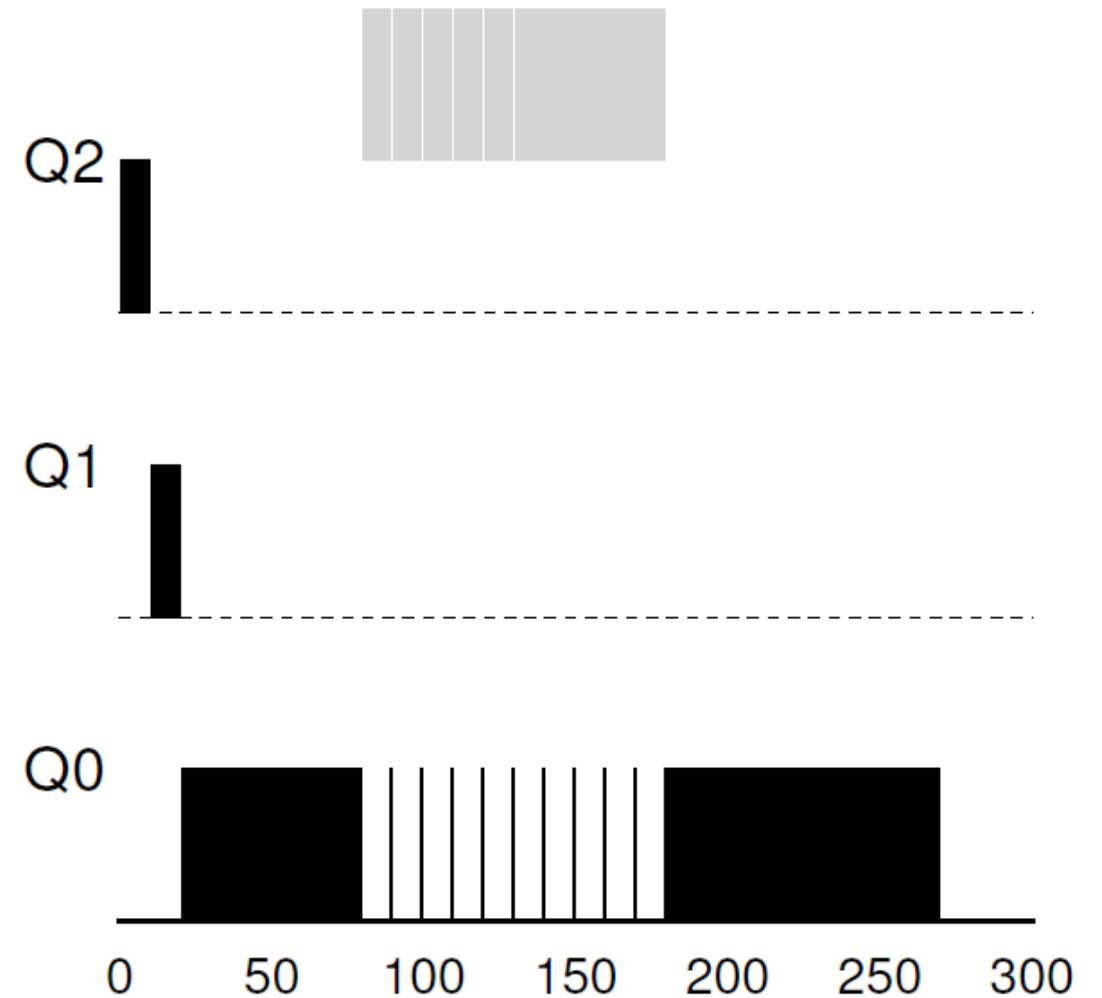
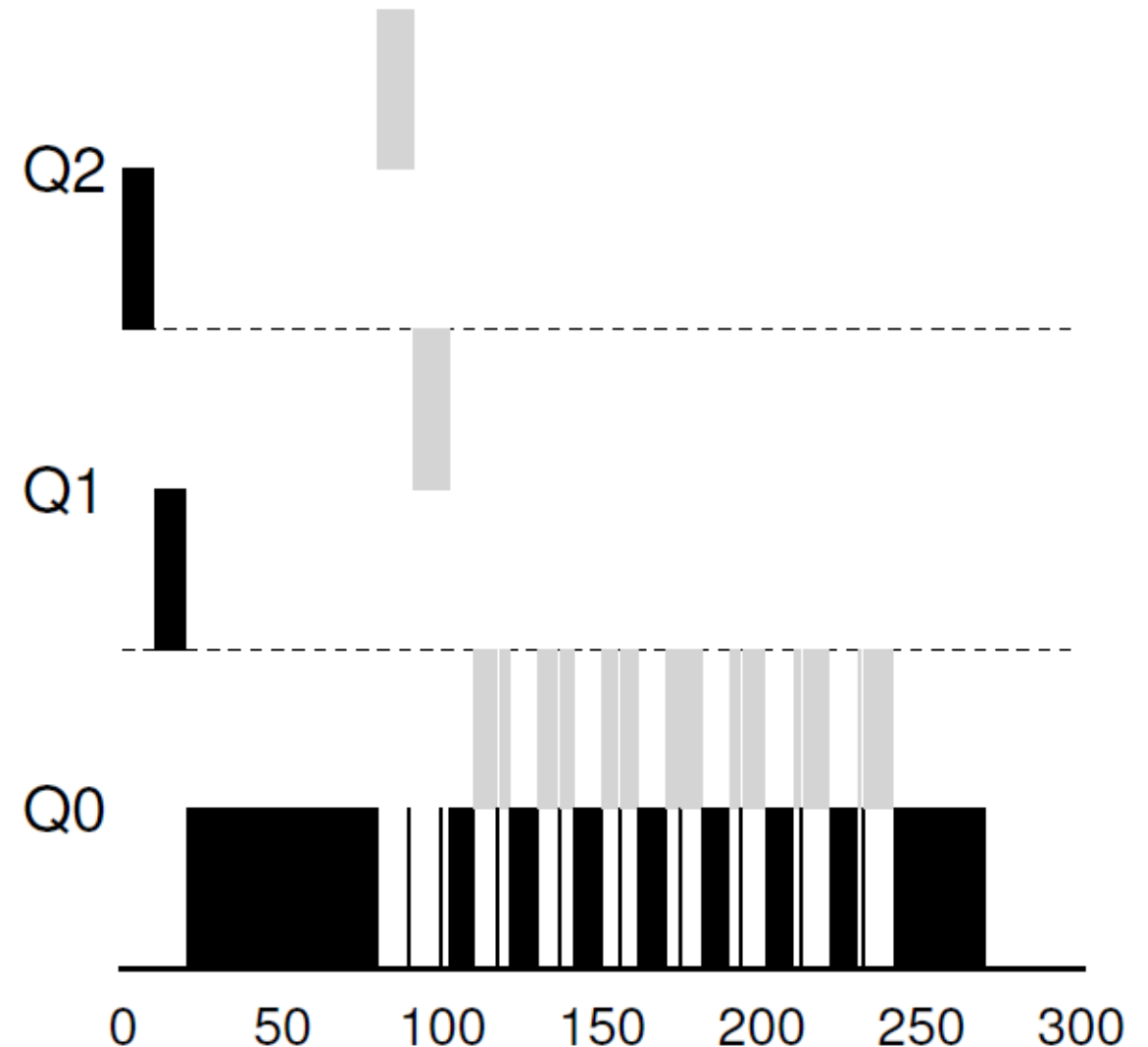


Figure 8.6: **Without (Left) and W**

Solution: better accounting

- Keep a record of how much CPU a process has consumed at a level
- Once it depletes its quota, lower its priority.



Tuning parameters

- How many queues
- How big a time slice
- What value for S (priority boost period)
- No easy answers.
- Variable time slices across queues
- Solaris MLFQ provides a table the administrator can change
 - 60 queues
 - Time slices: 20ms – x00ms
 - Priorities boosted every 1 sec

Tuning parameters contd.

- FreeBSD 4.3 scheduler
 - $\text{Priority}_i = f(\text{cpu_usage}_i)$
 - cpu_usage_i degrades over time (resulting in priority boost)
- Some systems reserve highest priority queues for OS jobs
- Some systems allow user advice (e.g. Linux nice utility)

Conclusion

- MLFQ tries to balance turnaround time and response time
- It does so with no prior knowledge of a job's running time
- It observes job behavior and learns and adjusts their priority